OxML 2025 Practical Tutorial:
Deep Learning and Representation Learning

Dr. Hao Liang
hao.liang@kcl.ac.uk
King's College London

Outline

- **Part I: Representation Learning Basics**
- **Part II: Deep Learning with MNIST and CIFAR-10**

Link and Slides

- Link for Part I
- Link for Part II

Part I: Representation Learning Basics

Representation Learning Basics

- **Definition:** Representation Learning extracts meaningful and informative features automatically from raw data.

- **Importance:** Essential for effective prediction tasks and simplifying downstream tasks.

- **Objectives for today:**
  - Grasp core concepts.
  - Practical experience with neural network building.
  - Understand various implementation frameworks.

Neural Network Basics

- **Structure Explained:**
  - Input Layer: Receives raw data.
  - Hidden Layers: Perform transformations.
  - Output Layer: Produces predictions.
- **Components:**
  - **Weights:** Parameters learned during training.
  - **Biases:** Allow shifting of activation functions.
- **Forward Propagation:** Data flows forward, passing through layers, transforming inputs into outputs.

Activation Functions

- **Why Activation Functions?**
  - Introduce non-linearities to capture complex patterns.
- **Detailed Examples:**
  - **Sigmoid:** Output between 0 and 1; suffers from saturation issues.
  - **Tanh:** Output between -1 and 1; centered at zero aiding gradient flow.
  - **ReLU (Rectified Linear Unit):** Outputs zero or linear activation, effective against vanishing gradient issues.

Loss Functions

- **Role of Loss Functions:** Quantify discrepancy between predictions and actual labels.
- **Types of Loss Functions:**
  - **Mean Squared Error (MSE):** Ideal for regression tasks, penalizes larger errors strongly.
  - **Cross-Entropy Loss:** Preferred for classification tasks, effectively guides gradient updates.
- **Choosing the Right Loss:** Dependent on task type and model goals.

Backpropagation & Gradient Descent

- **Backpropagation Explained:**
  - Employs chain rule to compute gradients efficiently.
  - Gradients propagate backward from output to input layers.
- **Gradient Descent Process:**
  - Gradients inform parameter updates to reduce loss.
  - Optimization strategies: adjusting learning rate, parameter initialization.

Coding Practical (NumPy Implementation)

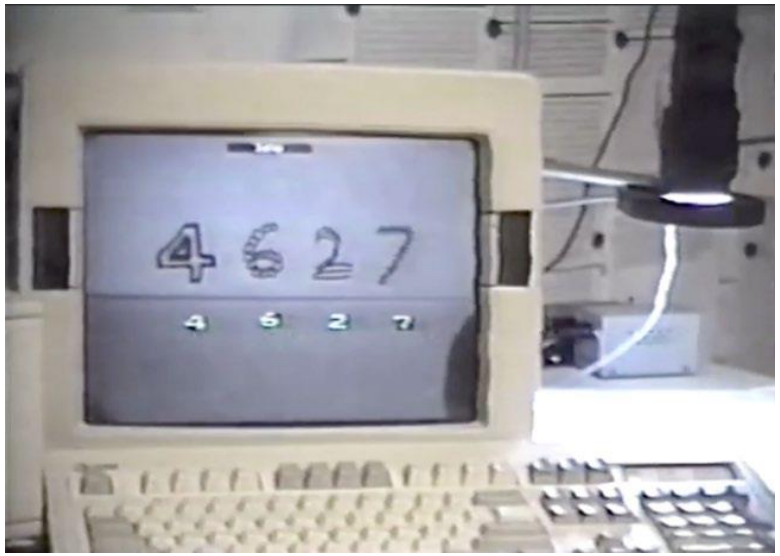
- **Hands-on Exercise: Solve XOR binary classification problem step-by-step.**

- **Visualization of Training Loss over Epochs**

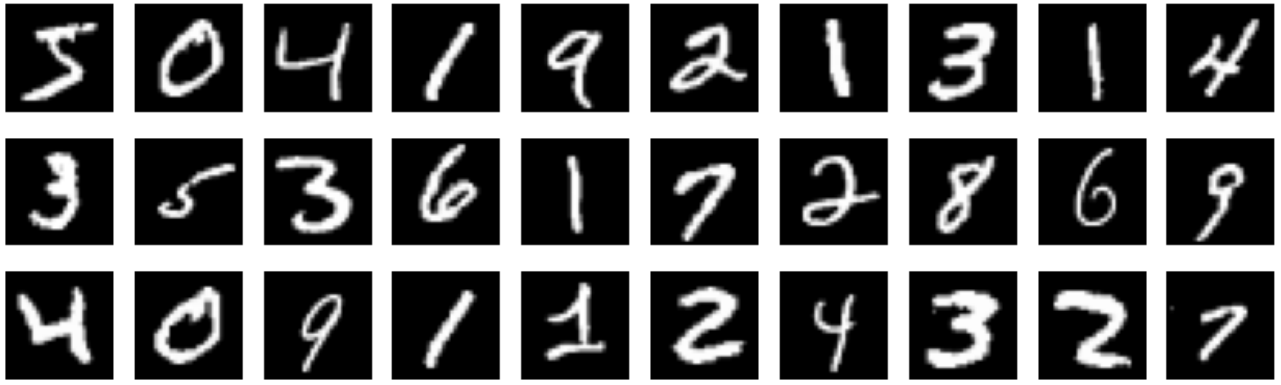
- **PyTorch Comparison**
  - Simplified code structure.
  - Built-in automatic differentiation for streamlined gradient computations.

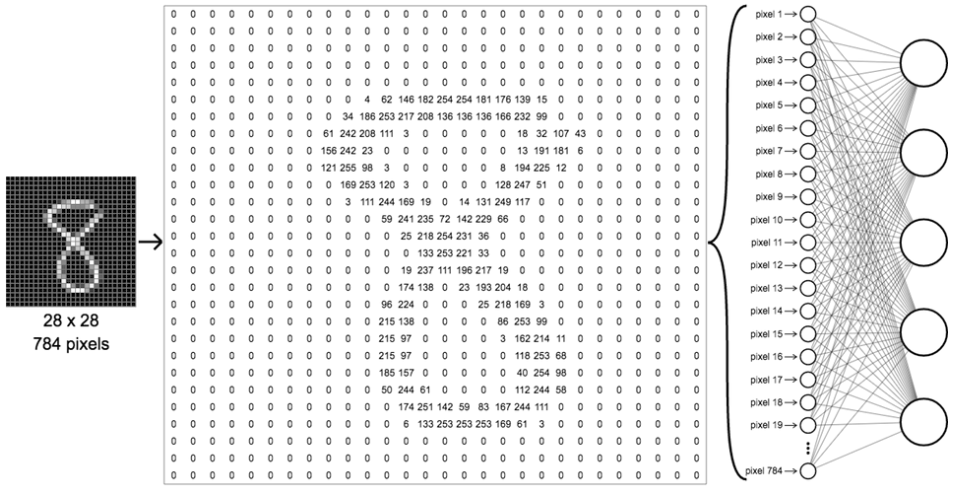
Part II: Deep Learning with MNIST and CIFAR-10

MNIST. Train a Classifier I



Problem: a real-world example of classification using neural networks, the task of recognizing and labeling images of handwritten digits. We are going to use the MNIST dataset, which contains **60,000** labeled images of handwritten digits sized 28x28 pixels, whose classification accuracy serves as a common benchmark in machine learning research.



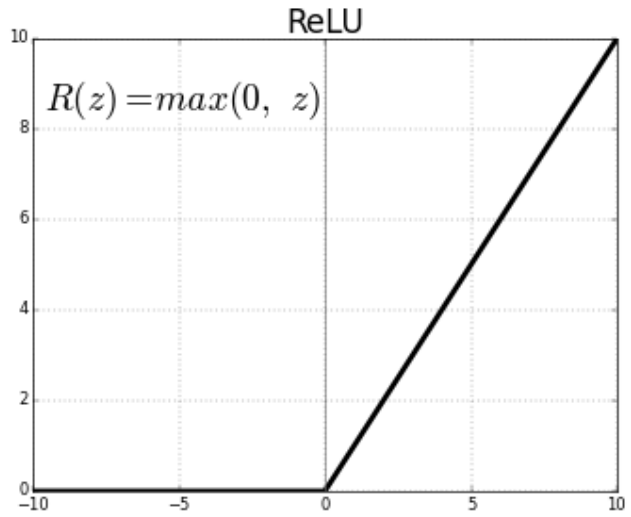


Remark: Activation Function

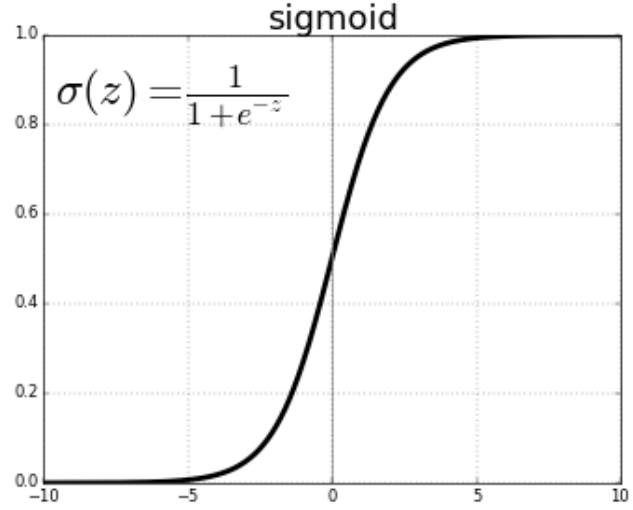
The activation function takes the same weighted sum input from before, $z = b + \sum_i w_i x_i$, and then transforms it once more before finally outputting it.

Advantages: Introducing Non-linearity / Increasing Model's Representational Power / Introducing Sparsity and Feature Selection / Providing Gradients for Backpropagation.

```
x = F.relu(self.fc1(x))
x = F.relu(self.fc2(x))
```




```
x = F.sigmoid(self.fc1(x))
x = F.sigmoid(self.fc2(x))
```



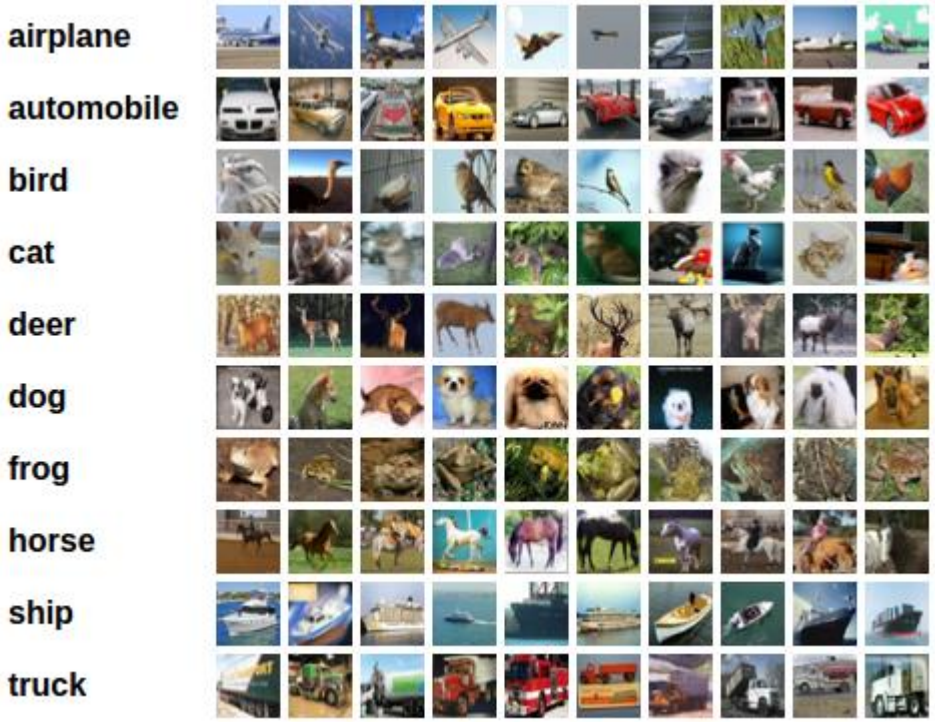
CIFAR10, Train a Classifier II

Problem: The CIFAR10 dataset, which has the classes: 'airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck'.

The images in CIFAR-10 are of size 3x32x32, i.e.

3-channel color images of 32x32 pixels in size.

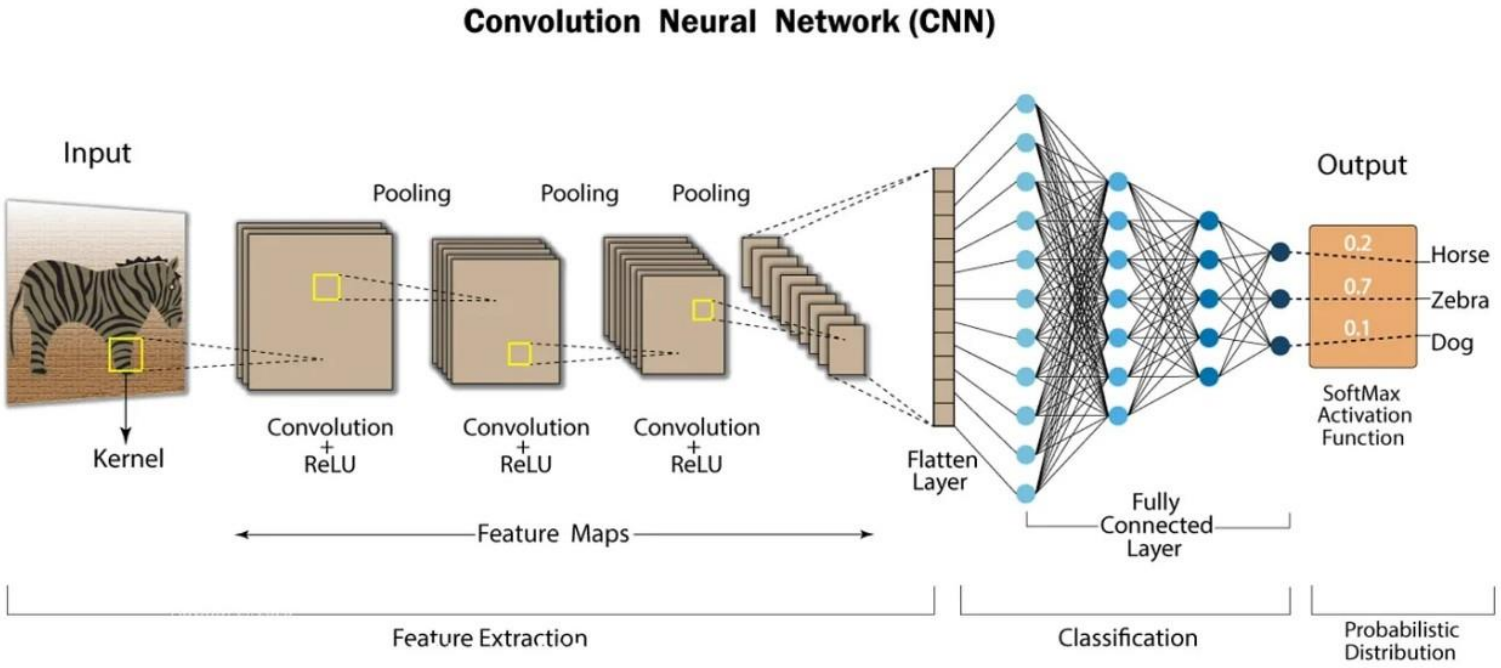
Training an image classifier to identify the type of the input image.



Remark: Convolutional Neural Network

What is the Convolutional Neural Network (CNN)?

CNN are a type of deep learning neural network architecture that is particularly well suited to image classification and object recognition tasks. A CNN works by transforming an input image into a feature map, which is then processed through multiple convolutional and pooling layers to produce a predicted output.




```python
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1) # flatten all dimensions except batch
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x


net = Net()
```